PRODUCT DOCUMENTATION

NetraOM
CSPM Trust Graph

A local, educational cloud-security-posture demo

Prepared for: Internship Assessment

Repository inspected on 10 October 2026

Version 1.0

Index

1. Project Overview . . . 3

2. Features and Status . . . 3

3. Technology Stack . . . 4

4. Architecture and Data Flow . . . 5

5. Attack-Path Algorithm . . . 6

6. Risk Scoring . . . 8

7. Detection Engine . . . 10

8. Cloud Collectors . . . 11

9. Database and API . . . 12

10. Setup and Testing . . . 14

11. Limitations and Security Notes . . . 15

12. Orca Security and Wiz Comparison . . . 16

13. Summary and Glossary . . . 17

Scope and evidence. This guide describes the repository as inspected on 10 October 2026. "Implemented" means the behavior is present in the source code. It does not mean that a live cloud account, production security, or every deployment path was tested.

1. Project Overview

NetraOM CSPM Trust Graph is a **local, educational cloud-security-posture demo**. It shows how inventory, configuration checks, graph relationships, attack-path analysis and findings can fit together. It is not a deployed commercial CSPM service.

Problem it demonstrates

A cloud configuration issue matters more when an attacker can reach it through network exposure and identity permissions, especially if the destination holds important data. The project uses:

- **PostgreSQL** for inventory and finding records that are easy to list, filter and inspect.
- **Neo4j** for a directed trust graph and neighborhood queries.
- **NetworkX**, an in-memory graph library, for local relationship and attack-path analysis.
- **A React UI and teaching labs** for inspecting the resulting records and graph.

Intended users

- Students and instructors learning CSPM, graph relationships and attack-path analysis.
- Developers evaluating an example FastAPI, PostgreSQL, Neo4j and React architecture.
- Security practitioners who want a small, fixture-driven prototype to explore.

Practical use case

Start the local services, load the AWS fixture and inspect a finding for world-open SSH. The demo shows the internet CIDR, the security group, the compute instance, the attached identity and a storage destination connected by permissions. The same finding is visible in the API, the PostgreSQL lab and the graph-oriented UI.

> **Data warning.** The checked-in fixtures are synthetic examples with placeholder account and resource identifiers. The demo load buttons in the main UI use these fixtures. They are not live AWS or GCP scans.

2. Features and Status

Feature	Status	What the repository does
Dashboard and inventory	Implemented (demo)	Shows the demo tenant's assets, facets and findings; supports fixture loading and asset detail.
Enterprise demo scenario	Implemented, synthetic	Generates a fixture of roughly 500 assets through `generate_enterprise_fixture`. It is not a real account scan.
AWS and GCP bundled scenarios	Implemented, synthetic	Loads checked-in normalized fixtures through `/demo/load`.
AWS and GCP cloud inventory	Implemented, limited	Optional read-only collectors run from a CLI, cover a limited resource set and write to the local stores. Their output is not shown in the regular demo API or UI.
CSPM misconfiguration checks	Implemented, rule-based	Checks selected public storage, network ingress, database, API Gateway, load balancer and Lambda conditions. It is not a full benchmark engine.
Toxic-combination detection	Implemented, bounded	Patterns T1 to T9, with only a subset enabled for small fixtures by default. D1 and D2 are extra demo patterns.
Attack-path discovery	Implemented, prototype	Uses an explicit traversal policy and a bounded priority-queue search (Section 5).
Four-factor risk score	Implemented, heuristic	Exposure, privilege, exploitability and impact are each scored 0 to 25 and added. The model is not calibrated or externally validated.

Feature	Status	What the repository does
Findings route evidence	Implemented	Findings store exact route nodes and edge IDs separately from graph-neighborhood context.
PostgreSQL and Cypher labs	Implemented, restricted	Read-oriented templates with validation guards. They are not general database consoles.
Threat map	Mocked / simulated	Uses finding-derived illustrative locations or a labeled fallback dataset. It is not a live threat feed.
Dashboard inventory history	Partial, browser-local	Stores daily counts in browser `localStorage` when the Dashboard is visited. It is not shared or retroactive.
Authentication and tenant UX	Missing	Routes use the configured demo tenant and expose no end-user login flow.
Auto-remediation	Missing	The UI may show guidance, but it does not change cloud resources or open tickets.
Compliance, monitoring, runtime detection	Missing	No full framework benchmark, event-driven scanner or workload sensor is implemented.

3. Technology Stack

Technology	Purpose in this repository
Python 3.11+	Backend and optional collector code.
FastAPI / Uvicorn	JSON API, interactive OpenAPI (Swagger) documentation and the local server.
Pydantic	Typed graph contract models and API request models.
SQLAlchemy + psycopg	PostgreSQL models and persistence.
PostgreSQL 16	Relational asset and finding tables.
Neo4j 5	Persistent property graph used by the demo and the graph query labs.
NetworkX MultiDiGraph	In-memory graph for combination analysis and attack-path traversal; keeps parallel edge IDs.
React 19, TypeScript, Vite	Single-page UI and build system.
@xyflow/react	Interactive graph canvas for asset and finding graphs.
Tailwind CSS 4 and Recharts	UI styling and dashboard charts.
Docker Compose	Local orchestration for Neo4j, PostgreSQL, API, Adminer and UI.
boto3 (optional)	AWS collector SDK; uses the standard boto3 credential chain.
Google Cloud libraries (optional)	GCP collector APIs; use Application Default Credentials (ADC).
pytest	Backend unit and integration-oriented tests.

4. Architecture and Data Flow

System architecture

Inputs are ingested and normalized by the backend. The same data is written to PostgreSQL and Neo4j and is also loaded into an in-memory NetworkX graph, which the detection and attack-path code reads. The UI and the teaching labs talk only to the API.

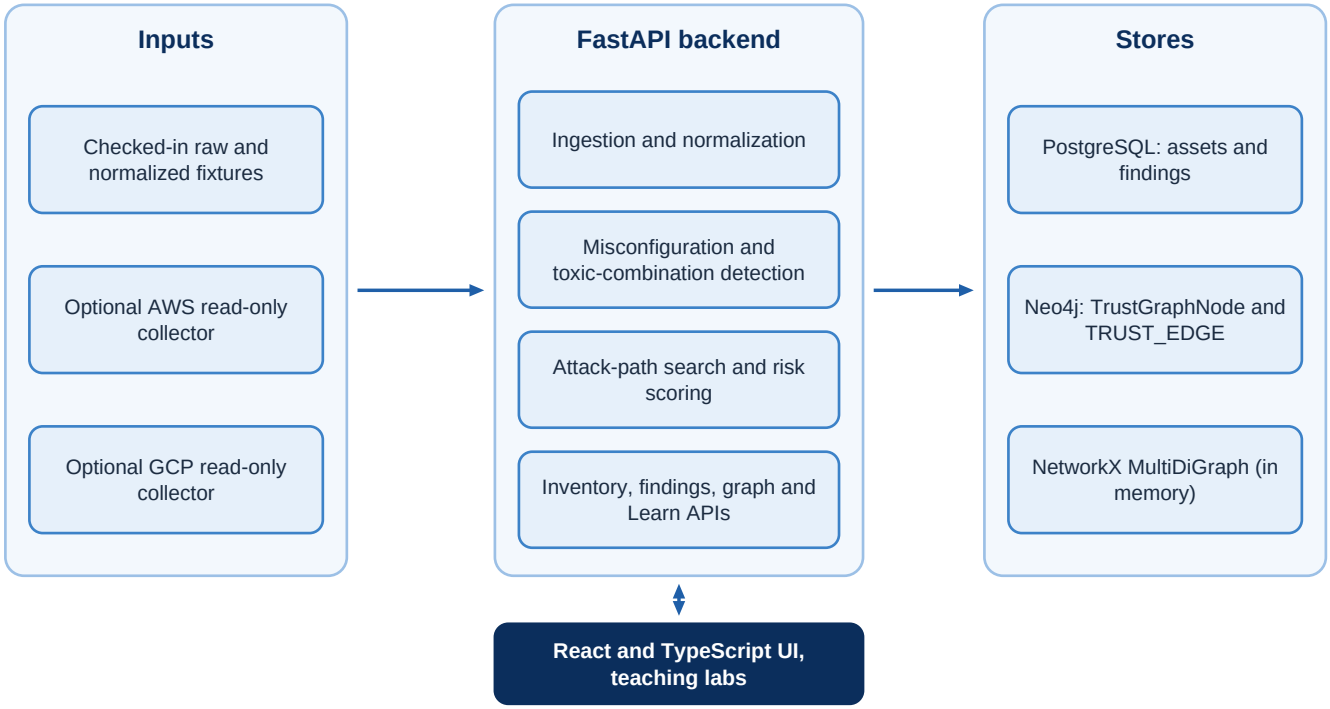


Figure 1. System architecture

End-to-end fixture flow

The sequence is visible in `demo_load()` and `_run_detection()` in `backend/app/main.py`, in `ingest_fixture_dict()` in `ingest.py`, and in `evaluate_and_persist_combos()` in `combos.py`.

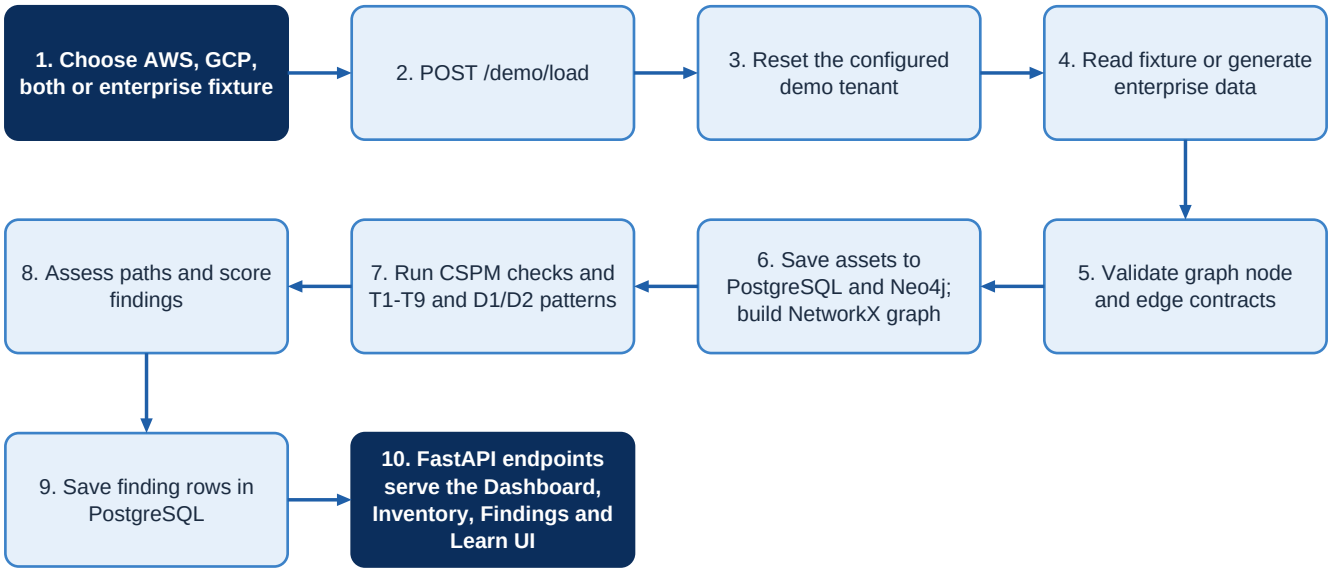


Figure 2. Fixture loading flow (read left to right, then right to left on the next row)

Important implementation details

- Fixture records are validated as `TrustGraphNode` and `TrustGraphEdge` contract models.
- Asset rows are upserted by tenant and graph node ID. The `graph_node_id` is the bridge to graph nodes.
- Neo4j stores nodes labeled `TrustGraphNode` and directed relationships typed `TRUST_EDGE`, with `kind`, `edge_id` and `props`.
- `NetworkXTrustGraph` uses a `MultiDiGraph` whose edge key is the edge ID, so parallel edges are retained.
- The cloud collector CLI feeds its normalized bundle through the same ingestion and detection path, but uses a distinct cloud tenant ID.

5. Attack-Path Algorithm

Graph model and traversal directions

Node kinds in the graph contract are account, identity, resource, network and external. Edges are directed and typed. The policy `ATTACK_TRAVERSAL_POLICY` in `backend/app/attack_path_engine.py` decides how each stored relationship is traversed. It does not rewrite the stored relationship.

Relationship kind	Attacker traversal policy
`in_vpc, in_subnet, attached_to, has_sg, exposes`	Reverse of the stored direction.
`owns, contains, uses, can_assume, can_read, can_write, can_admin, grants, trusts, federates_to, allows_ingress, allows_egress, resolves_to, serves, redirects_to, runs_image, deploys`	Forward stored direction, with special handling for `allows_ingress`.
Unknown relationship kinds	Not traversed by the attack-path engine.

For `allows_ingress`, a world or public endpoint can change the direction. A stored edge from a security group to `net:0.0.0.0/0` is traversed from the internet to the security group. A relationship already stored in attacker order stays forward. The helper `path_edges_for_route()` resolves every route edge by its exact edge ID, and `ordered_route_node_ids()` rebuilds node order only from a complete ordered route.

Search, bounds and ranking

The finding scorer calls `assess_finding_path()`, which works as follows:

1. It sorts and deduplicates the finding's node IDs.
2. It picks entry nodes among those finding nodes: IDs starting with `net:` or `ext:`, external-kind nodes, or nodes marked public or internet-exposed.
3. For every entry and every other finding node, it calls `find_rank_paths_dijkstra()`.
4. The search enumerates simple paths with a min-heap ordered by cumulative traversal cost. A path may not revisit a node.
5. It stops at the hop bound (default 8) and the candidate bound (default 10 per source-target search).
6. Candidates are scored independently, then ordered by descending risk score, ascending traversal cost, and finally node and edge IDs.
7. If no route is found, the finding is kept and receives the highest single-node score among its nodes. The unordered node collection is never described as a route.

Edge cost uses the edge-kind weight and the vulnerability weight of the destination node:

```
risk_factor = max(0.01, relationship_weight x destination_vulnerability_weight)
edge_cost   = -ln(risk_factor) + 0.05
path_cost   = sum of edge_cost for each edge
```

Lower costs enter the heap first. The implementation is a bounded, Dijkstra-style **simple-route enumerator**. It is not A* (there is no target heuristic), and BFS is not the search used for finding scoring, although a separate BFS candidate helper exists.

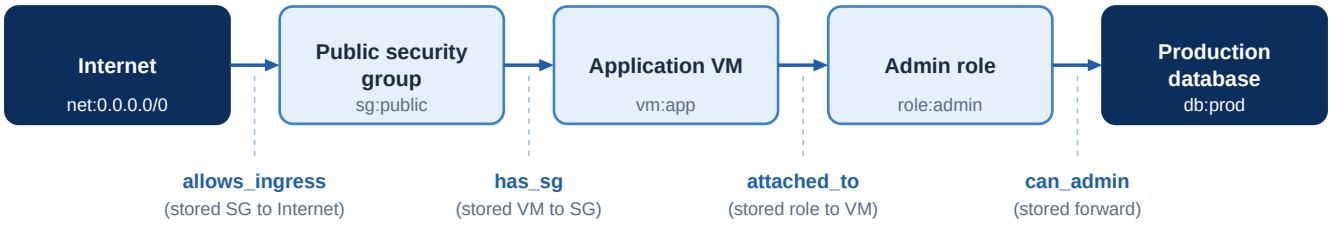
> **Important.** The candidate limit is applied while routes are discovered by traversal cost, before risk sorting. A high-risk route that is relatively expensive can therefore be excluded if more cheaper candidates are found first. Traversal cost is not the risk score.

Pseudocode

```
find_rank_paths(source, target):
    heap = [(cost=0, nodes=[source], edge_ids=[])]
    candidates = []
    while heap is not empty:
        route = pop route with smallest cumulative cost
        if route reaches target:
            keep route; stop if candidate limit reached; continue
        if route edge count reached max hops: continue
        for each allowed attacker-traversal edge (stable order):
            if next node already in route.nodes: continue
            push(route + edge, cost + edge cost)
    resolve exact edge IDs for each candidate
    score each candidate with four risk factors
    return candidates ordered by risk desc, cost asc, then IDs
```

Example route

This illustrative route follows the checked-in traversal policy. The ordered nodes are Internet, security group, VM, role and database. The route keeps one exact edge ID per hop, even when the traversal direction differs from the stored direction. Neighborhood edges are context only and are not traversed steps.



Arrows show attacker movement, which can differ from the stored edge direction.

Figure 3. Example attack route

Search flow

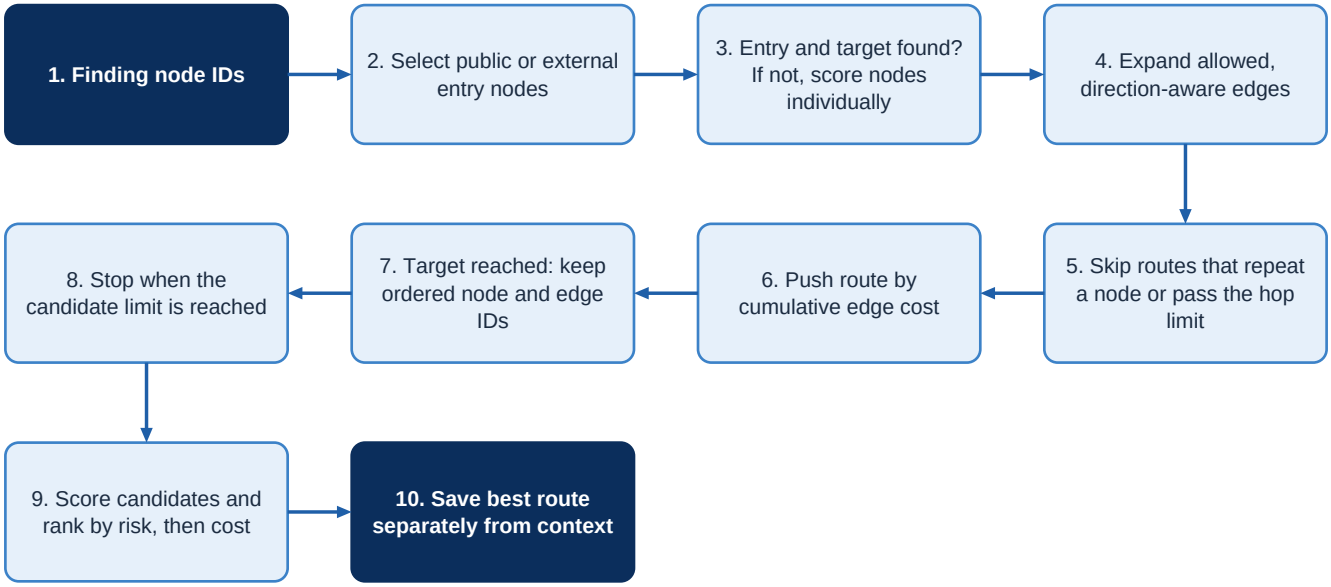


Figure 4. Attack-path search flow

6. Risk Scoring

The scorer computes four integer subscores, each limited to 0 to 25, and adds them. Subscores use Python `round()` and are clamped to the range.

```
final_score = min(100, exposure + privilege + exploitability + impact)
```

Score	Severity
90 to 100	Critical
70 to 89	High
40 to 69	Medium
0 to 39	Low

Factor	Calculation and inputs
Exposure	Take the maximum node exposure weight along the route. Multiply by 1.0 if the first route node is `net:` or marked public, otherwise by 0.7. Multiply by 25, round and clamp. The weight helper gives 1.0 to a node marked public or internet-exposed, 0.7 to `in_public_subnet`, and 0.2 otherwise. The configuration lists a separate 0.9 public weight, but the helper currently maps both flags to 1.0.
Privilege	Let P be the maximum node privilege weight and E the maximum relationship weight among `can_admin`, `can_write` and `can_assume` edges (0 if none). Score = round((0.7P + 0.3E) x 25). Node weights: admin 1.0, can-write 0.75, can-read 0.4, identity 0.5, otherwise 0.2.
Exploitability	Let V be the maximum vulnerability weight on route nodes and R the average relationship weight on route edges (0.8 if no edges). Score = round((0.5V + 0.5R) x 25). Severity weights: Critical 1.0, High 0.75, Medium 0.5, Low 0.25, None 0.0; an unrecognized severity uses 0.2.
Impact	Use the final route node as the target. Let C be its criticality weight and S its data-sensitivity weight. Score = round((0.6C + 0.4S) x 25). Criticality: crown-jewel 1.0, production 0.85, staging 0.5, development 0.2, default 0.3. Sensitivity: critical-data or PII 1.0, confidential 0.7, internal 0.3, otherwise 0.0.

Weights are centralized in `backend/app/risk_config.py`. The calculation and severity thresholds are in `evaluate_path_risk_breakdown()` in `attack_path_engine.py`.

Worked example

Assume the route in Section 5 has an internet entry, a Critical-severity VM, an admin-capable role with a `can_admin` edge, and a crown-jewel database marked critical data. The edge kinds are `allows_ingress` (0.95), `has_sg` (default 0.50), `attached_to` (0.85) and `can_admin` (0.95).

```
Exposure       = round(1.00 x 1.00 x 25)                    = 25
Privilege      = round((1.00 x 0.70 + 0.95 x 0.30) x 25)    = 25
Average edge R = (0.95 + 0.50 + 0.85 + 0.95) / 4            = 0.8125
Exploitability = round((1.00 x 0.50 + 0.8125 x 0.50) x 25)  = 23
Impact         = round((1.00 x 0.60 + 1.00 x 0.40) x 25)    = 25
Final          = min(100, 25 + 25 + 23 + 25)                = 98 (CRITICAL)
```

> **Note.** This is a transparent example calculation. It is not a measured production calibration.

Risk-scoring flow

The input is the selected exact route, or the finding nodes when no route exists.

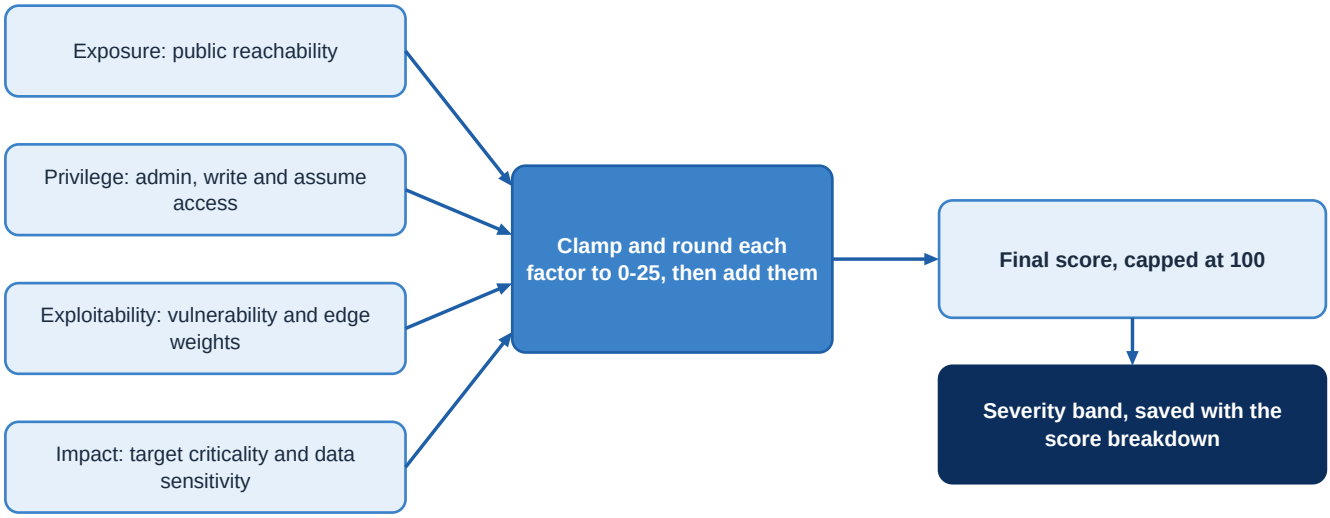


Figure 5. Risk-scoring flow

Risk score and traversal cost answer different questions. The score is the finding-risk heuristic, while the cost only orders route candidates during the search.

7. Detection Engine

CSPM misconfiguration rules

`evaluate_misconfigs()` in `backend/app/misconfig.py` evaluates the records it receives and saves one deduplicated finding per fingerprint.

Rule IDs	Condition represented
`aws_s3_public_read`, `gcp_storage_public_read`	S3 or GCS storage is public or has anonymous read or write metadata.
`storage_public_anonymous_write`	S3 or GCS storage is marked anonymous-write.
`storage_public_unencrypted`	Public S3 or GCS storage is explicitly marked unencrypted.
`aws_apigw_public_no_auth`	API Gateway authorization is `NONE`.
`aws_alb_public_internet`, `aws_nlb_public_internet`	Load balancer is internet-facing or public.
`aws_rds_public_exposure`	RDS instance or cluster is publicly reachable.
`aws_lambda_public_invoke`	Lambda is marked public or internet-exposed.
`aws_sg_world_ssh`, `gcp_firewall_world_ssh`	Ingress from a world CIDR on port 22.
`aws_sg_world_rdp`	Ingress from a world CIDR on port 3389.
`aws_sg_world_web`	Ingress from a world CIDR on port 80 or 443.

These are selected property checks, not exhaustive provider policy coverage. A missing property can behave differently from an explicit false value. For example, T6 treats a public resource as unencrypted unless `is_encrypted` is exactly true.

Toxic combinations

`PATTERN_REGISTRY` in the vendored `netraom-graph-writer` package defines nine patterns:

ID	High-level pattern
T1	Public writable storage and a write-capable identity.
T2	Internet SSH ingress to compute, an attached role, and writable public or critical storage.
T3	A resource exposes an external surface.
T4	Cross-cloud federation or assumption relationship.
T5	Internet-reachable compute with an attached role that can access critical data.
T6	Public storage without an explicit `is_encrypted: true`.
T7	Public RDS or Redshift database.
T8	Public function with a role that can read secrets.
T9	Internet-reachable compute with a role that has a `can_admin` edge.

The normal `/demo/load` flow selects only **T1, T2, T5, T6 and T9** for small AWS or GCP fixtures. The `both` and `enterprise` options request all of T1 to T9. The enterprise demo also enables **D1** (unauthenticated API Gateway to sensitive writable data) and **D2** (public load balancer or exposed compute to exposed data). The cloud collector CLI enables all patterns and the demo patterns.

Patterns are heuristics tied to specific node IDs, properties and edge conventions. Some shared helpers match AWS-style role IDs and may not recognize every GCP service-account naming shape, so a passing test fixture does

not prove complete cross-cloud coverage.

Finding creation and fingerprints

- Misconfiguration fingerprints are stable hashes of the rule ID and a canonical key. Non-demo tenant IDs are included in the hash.
- Toxic-combination fingerprints are stable hashes of the pattern ID and the sorted matching node IDs. Non-demo fingerprints are tenant-scoped.
- Findings are assessed against the in-memory graph before they are saved, and the path and risk breakdown are stored on the finding row.
- `trust_graph_node_ids` lists finding-related nodes. `path_edges`, `attack_path_nodes` and `attack_path_edge_ids` hold the selected ordered path when one exists.

8. Cloud Collectors

Collectors are optional modules run explicitly with `python -m app.cloud_collectors.cli`. They are not called by the UI buttons or by a scheduler. Provider API access is read-only, and collected records are written to the application's PostgreSQL and Neo4j stores.

Cloud	Implemented resource scope
AWS	S3 bucket list, location, policy, policy status, public access block, ACL and encryption configuration; EC2 instances, security groups, ingress rules and subnet, security group and instance-profile relationships; IAM roles, attached role policies, instance profiles, and IAM policy simulation for selected S3 object read and write actions. EC2 uses the selected region, while S3 and IAM are account-level.
GCP	Cloud Storage buckets and bucket IAM bindings; Compute Engine zones, instances, ingress firewall rules and target matching; service accounts and the project IAM policy. Role mapping is limited to selected Cloud Storage reader, writer and admin roles.

Authentication and permissions

- AWS calls use `boto3.Session()` without explicit credentials, relying on the standard credential chain. Set the region with `AWS_DEFAULT_REGION` or `--region`.
- GCP uses Application Default Credentials through `google.auth.default()` with a read-only Cloud Platform scope. A project ID is required.
- The collectors do not accept cloud secrets as CLI parameters. Detailed permission lists are in `backend/README.md`.
- Cloud records use `CLOUD_TENANT_ID` (default `22222222-2222-4222-8222-222222222222`), which is distinct from `DEMO_TENANT_ID`. This protects them from the demo reset and load flow. However, the standard asset and finding APIs filter to the demo tenant, so collected data is not shown in the UI.

Collector limitations

- Repeated collection upserts by stable IDs and does not remove resources that have disappeared from the provider scope.
- AWS IAM simulation covers only selected S3 object read and write decisions for buckets in scope. It does not enumerate all effective permissions.
- AWS account-level S3 Block Public Access and bucket policy conditions are not evaluated. Missing bucket encryption configuration stays unknown.
- GCP public exposure is an approximation from public IPs and matching enabled ingress rules. Firewall deny and priority interactions, inherited Public Access Prevention and conditional IAM bindings are not evaluated.
- GCP service-account OAuth scopes are recorded but do not constrain generated IAM permission edges.

- Collector tests use mocks. They do not prove live credentials, permissions, quotas or correctness for every resource configuration.

9. Database and API

PostgreSQL tables

The schema is defined in `docker/postgres/01-schema.sql` and mapped in `backend/app/db/models.py`.

Table	Important fields	Purpose
assets	asset_id, tenant_id, graph_node_id, canonical_key, cloud, type, region, is_internet_exposed, props, last_seen	Portal-style inventory. The pair (tenant_id, graph_node_id) is the logical unique key.
findings	fingerprint, tenant, source, rule, title, severity, trust_graph_node_ids, path_edges, risk_score, score_breakdown, status, last_seen	CSPM and toxic-combination results, with an optional route and factor breakdown.

There is no inventory history or snapshot table. The Dashboard history chart is browser-local.

Neo4j and the graph contract

- Nodes use the label `TrustGraphNode` with tenant and node IDs, kind, name, cloud, resource type, region and serialized properties.
- Edges are directed `TRUST_EDGE` relationships with `edge_id`, `kind` and properties.
- The startup handler creates a uniqueness constraint on `(tenant_id, node_id)`.
- The contract models and enums are in the vendored `netraom-contracts` package.
- The in-memory `NetworkXTrustGraph` is a `MultiDiGraph` and is not the Neo4j database.

Main API routes

These routes are declared in `backend/app/main.py`. The teaching `/learn` router is hidden from the generated OpenAPI schema.

Route	Purpose
GET /health	Basic process health response.
POST /demo/load	Reset and load one of aws, gcp, both or enterprise into the demo tenant, then run detections.
POST /demo/reset	Clear the demo tenant's graph, assets and findings.
GET /assets/facets, GET /assets	Inventory counts and facets; paginated and filtered assets.
GET /assets/by-node/{graph_node_id}, GET /assets/{asset_id}, GET /assets/{asset_id}/json	Look up an asset, including its normalized, raw and graph representation where available.
GET /graph/nodes/{node_id}/neighborhood, GET /graph/schema	Read a graph neighborhood or the supported graph kind names.
GET /findings/facets, GET /findings	Finding counts, filtering, sorting and pagination.
GET /findings/{fingerprint}/path	Return the finding, graph context, related assets, exact ordered route nodes and edges, and score data.
/learn/* endpoints	Teaching fixture, statistics, SQL and Cypher endpoints.

The path response returns both `edges` (neighborhood context) and `attack_path_edges` (the selected route). Consumers should use the exact path arrays for route highlighting.

Frontend pages

Routes are defined in `ui/src/App.tsx`.

Page	Route	Purpose
Dashboard	/	Fixture loading, inventory and finding metrics, charts and recent findings.
Inventory	/assets	Search and filter assets.
Asset detail	/assets/:assetId	Asset properties and graph neighborhood.
Findings	/findings	CSPM and toxic-combination findings with their route and context graph.
Threat map	/threats	Simulated, illustrative threat scenarios.
Learn	/learn	Teaching overview.
Raw to normalized	/learn/inventory	Fixture normalization lab.
Postgres lab	/learn/postgres	SQL examples and a read-restricted query lab.
Graph lab	/learn/graph	Cypher examples and query lab.
Cookbook	/learn/cookbook	Guided examples.
Attack paths	/learn/attack-path	Educational attack-path workflow and illustrative route.
Assistant	/assistant	Rule-based local FAQ matching. It is not an LLM and makes no AI API call.

10. Setup and Testing

Run the local demo

Requirements: Docker Desktop (or Docker with Compose v2), a browser and roughly 4 GB of free RAM.

```
docker compose up -d --build
curl -X POST http://localhost:8000/demo/load \
  -H "Content-Type: application/json" \
  -d '{"fixture":"enterprise"}'
```

Use `aws`, `gcp` or `both` for smaller fixtures. The Dashboard also has buttons for the enterprise, small AWS and AWS plus GCP demo data.

Service	Local URL and access
React UI	http://localhost:5173
FastAPI Swagger	http://localhost:8000/docs
FastAPI ReDoc	http://localhost:8000/redoc
Adminer	http://localhost:8081 (server postgres, database cspm_demo, read-only user student, password student)
Neo4j Browser	http://localhost:7474 (local lab login neo4j / changeme)
PostgreSQL host port	localhost:5433

> **Caution.** The Compose credentials are for a local teaching environment only. Do not reuse them in a deployed system.

Run the cloud collectors

These are separate commands and are not part of the fixture-loading buttons. Install the optional SDK dependencies, provide valid provider authentication, and keep the local PostgreSQL and Neo4j services running.

```
cd backend
pip install -e ".[dev,cloud]"
python -m app.cloud_collectors.cli aws --region us-east-1
python -m app.cloud_collectors.cli gcp --project-id my-gcp-project
```

Tests and builds

```
# Backend tests (from the repository root)
pytest tests/

# Frontend type-check, build and lint
cd ui
npm ci
npm run build
npm run lint
```

Existing tests cover attack-path traversal and scoring, exact edge ordering, fixture detection, mocked AWS and GCP collector output, NetworkX ingestion, SQL and Cypher guards, and the generated enterprise fixture shape. The tests `test_api_risk_fields` and `tests/test_performance_scale.py` call the local API, so the API and its backing services must be running. Collector tests use mocks and need no real cloud credentials.

> **Verification record.** Source code, test definitions, fixtures, schemas and configuration were inspected. No tests or builds were run for this document. The behaviors described are therefore source-backed, but current runtime health, API performance, live cloud permission sufficiency and cloud integration are not newly verified.

11. Limitations and Security Notes

Functional limitations

- The main workflow is demo-tenant and fixture-centered. It is not a production multi-tenant SaaS.
- Collectors cover only the listed resources and do not collect continuously.
- Cloud collector results use a separate tenant, but the main API and UI queries are hard-coded to the demo tenant.
- Ingestion is retry-friendly and upsert-based, but it does not remove stale resources that have vanished from the cloud.
- Attack paths depend on the supplied finding node IDs, supported relationship kinds, graph metadata and search bounds. The engine does not search every graph node as an arbitrary source and target pair.
- Candidate pruning is by traversal cost, before the final risk sorting.
- Risk weights are fixed heuristics. Several fields depend on the provider or fixture, and missing metadata changes scores.
- Rule checks are hand-coded and do not implement full AWS or GCP policy semantics.
- The threat map is simulated. Inventory-over-time is browser-local and starts when a Dashboard visit records a daily snapshot. No schema supports server-shared history.

Security and operational cautions

- This is a **local demo, not a secure production deployment**. Compose uses simple local passwords. The FastAPI app has permissive CORS (`allow_origins=["*"]` with credentials enabled) and no authentication or authorization layer.
- `/demo/reset` deletes the demo tenant's graph, assets and findings, and `/demo/load` calls that reset. Do not expose these routes to untrusted users or production networks.
- The SQL and Cypher teaching endpoints have validation and tenant constraints, but this is application logic. It is not a substitute for an isolated database account and proper authorization. The PostgreSQL `student` role is SELECT-only in the provided setup.
- Attack-path scores and detector outputs are not proof of exploitability. Check provider state and policy semantics before using them for real security decisions.

12. Orca Security and Wiz Comparison

This section compares the **inspected repository** with the vendors' own public pages, checked on 10 October 2026. The vendor claims are not independently tested.

Capability	NetraOM (inspected)	Orca (public documentation)	Wiz (public documentation)
Cloud asset visibility	Synthetic fixtures plus limited, manually run read-only collectors. Not continuous; the UI reads the demo tenant.	Describes continuous, agentless visibility across clouds, workloads, identities, data, AI and code.	Describes continuous discovery through provider APIs and unified cloud visibility.
Misconfiguration coverage	Selected hand-coded checks plus graph combination rules. No full CIS, NIST or PCI engine.	Describes posture checks and mappings to frameworks such as PCI DSS, HIPAA, SOC 2, CIS, NIST and ISO.	Describes CSPM monitoring, risk prioritization and compliance frameworks.
Graph and attack paths	NetworkX in-memory graph, explicit traversal policy, bounded simple routes and exact edge IDs. Neo4j stores demo graph data.	Describes attack paths to critical assets built from exposure, identities, vulnerabilities and connections. Implementation not published.	Describes a Security Graph linking assets, identities, permissions, vulnerabilities and risks.
Risk prioritization	Visible four-factor 0 to 100 heuristic with fixed weights. Not a validated probability of compromise.	Describes scoring from reachability, asset value, exposure and context. No formula published.	Describes contextual risk from exposure, permissions, sensitive data and attack paths. No numeric formula published.
Workload and runtime coverage	No vulnerability scanner, workload agent or runtime detector.	Documents vulnerability reachability and workload and runtime capabilities.	Describes runtime signals and sensor-assisted validation in its wider product.
Remediation and workflow	Findings API and UI with analyst guidance. No cloud write actions or auto-remediation.	Markets integration and remediation workflows. Not verified here.	Describes linking paths to code and remediation workflows. No customer deployment claimed.

Official pages consulted

- Orca: orca.security/platform/reduce-cloud-risk, orca.security/platform/risk-prioritization and orca.security/platform
- Wiz: wiz.io/academy/cloud-security/what-is-cloud-security-posture-management-cspm and wiz.io/academy/detection-and-response/attack-path-analysis

> **Bottom line.** NetraOM is a small educational prototype that illustrates selected CSPM patterns and attack-path concepts. Orca and Wiz publicly describe broader commercial platforms, and this repository should not be presented as feature-equivalent to either product.

13. Summary and Glossary

Summary

The project demonstrates a useful end-to-end teaching workflow. It loads synthetic fixtures or selected read-only cloud inventory, normalizes resources into a typed graph, detects selected misconfigurations and toxic combinations, scores findings, and exposes them through APIs and a React UI. The graph and finding model keep exact ordered route evidence separate from neighborhood context.

It remains a local prototype. The rule catalog and collectors are limited, cloud inventory is not shown in the default demo tenant, the risk formula is heuristic, and live provider integrations were not verified for this document.

Glossary

Term	Simple meaning
Asset	A cloud resource in the inventory, such as a VM, bucket or database.
CSPM	Cloud Security Posture Management: checking cloud configuration for security problems.
Finding	A stored alert or result created by a detection rule.
Toxic combination	Separate facts or issues that together form a more serious scenario.
Trust graph	A graph of cloud entities (nodes) and their relationships (edges).
Node	One graph entity, such as an identity or a resource.
Edge / relationship	A typed, directed link between two graph nodes.
Canonical direction	The direction in which an edge is stored. It can differ from attacker movement.
Attack path	An ordered route through allowed relationships from an entry point toward a target.
Traversal cost	The engine's additive search weight for queueing route candidates. It is not the risk score.
Risk subscore	One 0 to 25 component: exposure, privilege, exploitability or impact.
Fingerprint	A stable identifier used to recognize the same finding across repeated scans.
Tenant	An identifier used to scope graph and relational records.
Fixture	A checked-in example dataset. The repository's demo fixtures are synthetic.
ADC	Application Default Credentials, Google Cloud's standard credential discovery mechanism.
MultiDiGraph	A directed graph structure that allows several distinct edges between the same pair of nodes.
APOC	Neo4j's optional procedure library, used for one neighborhood query path; a fallback traversal exists.